

# **Assignment-2**

**Name:-** Samiksha Sanjay Adake

**Roll Number:-** 2070120

**CS-A**

**PRN:-** 1251070524

## **Aim:**

Develop a program to identify and efficiently store a sparse matrix using compact representation and perform basic operations like display and simple transpose.

## **Objective:**

- To identify whether a given matrix is a sparse matrix by counting the number of zero and non-zero elements.
- To store the sparse matrix efficiently using compact (3-column) representation.
- To display the compact representation of the sparse matrix.
- To perform the simple transpose operation on the compact representation.
- To understand the concept of memory optimization using sparse matrix representation.

## **Theory:**

A sparse matrix is a matrix in which most of the elements are zero. Instead of storing all the elements, a compact representation stores only the non-zero elements, reducing memory usage.

In compact representation, the first row stores the number of rows, columns, and non-zero elements. The remaining rows store the row index, column index, and value of each non-zero element.

The program checks whether the given matrix is sparse, converts it into compact representation, displays it, and performs a simple transpose by interchanging the row and column indices of the non-zero elements. This method makes storage and processing more efficient.

## **Algorithm:**

### **Algorithm 1: Check Sparse Matrix**

1. Start.
2. Read the number of rows and columns.
3. Input all the elements of the matrix.
4. Count the number of zero and non-zero elements.
5. If the number of zero elements is greater than the number of non-zero elements, declare the matrix as sparse.
6. Otherwise, declare it as a normal matrix.

7. Stop.

### **Algorithm 2: Compact Representation**

1. Start.
2. Read the sparse matrix.
3. Store the total number of rows, columns, and non-zero elements in the first row of the compact matrix.
4. Traverse the original matrix.
5. For every non-zero element:
  - Store its row index.
  - Store its column index.
  - Store its value.
6. Display the compact representation.
7. Stop.

### **Algorithm 3: Display Compact Matrix**

1. Start.
2. Read the compact representation.
3. Display the first row containing rows, columns, and number of non-zero elements.
4. Display each row containing the row index, column index, and value of every non-zero element.
5. Stop.

### **Algorithm 4: Simple Transpose**

1. Start.
2. Read the compact representation.
3. Interchange the row and column indices of each non-zero element.
4. Update the first row by swapping the number of rows and columns.
5. Display the transposed compact representation.
6. Stop.

## Code:

```
#include <iostream>
using namespace std;

int rows, cols;
int matrix[100][100];
int sparse[101][3];
int nonZero = 0;
bool entered = false;

void enterMatrix()
{
    cout << "Enter the number of rows: ";
    cin >> rows;

    cout << "Enter the number of columns: ";
    cin >> cols;

    cout << "\nEnter the elements of the matrix:\n";

    nonZero = 0;

    for(int i=0;i<rows;i++)
    {
        for(int j=0;j<cols;j++)
        {
            cin >> matrix[i][j];

            if(matrix[i][j] != 0)
                nonZero++;
        }
    }

    int zero = (rows * cols) - nonZero;

    if(zero > nonZero)
```

```

        cout << "\nThe given matrix is a Sparse Matrix.\n";
    else
        cout << "\nThe given matrix is NOT a Sparse Matrix.\n";

    sparse[0][0] = rows;
    sparse[0][1] = cols;
    sparse[0][2] = nonZero;

    int k = 1;

    for(int i=0;i<rows;i++)
    {
        for(int j=0;j<cols;j++)
        {
            if(matrix[i][j] != 0)
            {
                sparse[k][0] = i;
                sparse[k][1] = j;
                sparse[k][2] = matrix[i][j];
                k++;
            }
        }
    }

    entered = true;
}

void displaySparse()
{
    if(!entered)
    {
        cout << "\nPlease enter the matrix first.\n";
        return;
    }

    cout << "\nSparse Matrix (3-Tuple Representation)\n";
    cout << "Row\tCol\tValue\n";

    for(int i=0;i<=nonZero;i++)

```

```

    {
        cout << sparse[i][0] << "\t"
              << sparse[i][1] << "\t"
              << sparse[i][2] << endl;
    }
}

void simpleTranspose()
{
    if(!entered)
    {
        cout << "\nPlease enter the matrix first.\n";
        return;
    }

    int transpose[101][3];

    transpose[0][0] = cols;
    transpose[0][1] = rows;
    transpose[0][2] = nonZero;

    int k = 1;

    for(int col=0; col<cols; col++)
    {
        for(int i=1; i<=nonZero; i++)
        {
            if(sparse[i][1] == col)
            {
                transpose[k][0] = sparse[i][1];
                transpose[k][1] = sparse[i][0];
                transpose[k][2] = sparse[i][2];
                k++;
            }
        }
    }

    cout << "\nSimple Transpose (3-Tuple Representation)\n";
    cout << "Row\tCol\tValue\n";

```

```

        for(int i=0;i<=nonZero;i++)
        {
            cout << transpose[i][0] << "\t"
                << transpose[i][1] << "\t"
                << transpose[i][2] << endl;
        }
    }
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== MENU =====\n";
        cout << "1. Enter Matrix\n";
        cout << "2. Display Sparse Matrix\n";
        cout << "3. Simple Transpose\n";
        cout << "4. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        if(choice == 1)
        {
            enterMatrix();
        }
        else if(choice == 2)
        {
            displaySparse();
        }
        else if(choice == 3)
        {
            simpleTranspose();
        }
        else if(choice == 4)
        {
            cout << "\nExiting Program...\n";
        }
    }
}

```

```

        else
        {
            cout << "\nInvalid Choice!\n";
        }

    } while(choice != 4);

    return 0;
}

```

## Output:

```

PS C:\Users\LOQ\OneDrive\Desktop\SY Preparation\college notes\DS\Assignmentsss> & 'c:\Users\LOQ\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\windowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-booie2yw.xpv' '--stdout=Microsoft-MIEngine-Out-yuo1xd4m.2yj' '--stderr=Microsoft-MIEngine-Error-ggdle0z5.hra' '--pid=Microsoft-MIEngine-Pid-mjqvhto4.czv' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'

```

```

===== MENU =====

```

1. Enter Matrix
2. Display Sparse Matrix
3. Simple Transpose
4. Exit

```

Enter your choice: 1

```

```

Enter the number of rows: 3

```

```

Enter the number of columns: 3

```

```

Enter the elements of the matrix:

```

```

1 2 3 4 5 6 7 8 9

```

```

The given matrix is NOT a Sparse Matrix.

```

```

===== MENU =====
1. Enter Matrix
2. Display Sparse Matrix
3. Simple Transpose
4. Exit
Enter your choice: 2

Sparse Matrix (3-Tuple Representation)
Row    Col    Value
3      3      9
0      0      1
0      1      2
0      2      3
1      0      4
1      1      5
1      2      6
2      0      7
2      1      8
2      2      9

===== MENU =====

```

```

===== MENU =====
1. Enter Matrix
2. Display Sparse Matrix
3. Simple Transpose
4. Exit
Enter your choice: 3

Simple Transpose (3-Tuple Representation)
Row    Col    Value
3      3      9
0      0      1
0      1      4
0      2      7
1      0      2
1      1      5
1      2      8
2      0      3
2      1      6
2      2      9

```

```

===== MENU =====
1. Enter Matrix
2. Display Sparse Matrix
3. Simple Transpose
4. Exit
Enter your choice: 4

```

```

Exiting Program...
PS C:\Users\LOQ\OneDrive\Desktop\SY Preparation\college notes\DS\Assignmentsss>

```

**Conclusion:**

The program successfully identifies whether the given matrix is a sparse matrix and stores it using the compact (3-tuple) representation. It efficiently displays the compact form and performs the simple transpose operation by interchanging the row and column indices of the non-zero elements. Using compact representation reduces memory usage by storing only the non-zero elements instead of the entire matrix. This program helps in understanding the concept of sparse matrices and their efficient storage and manipulation in C++.